
UNIT 3 CONTROL STRUCTURES, DATA STRUCTURES-AND LINKED LISTS, QUEUES

Structure

- 3.1 Introduction
- 3.2 Objectives
- 3.3 Terminologies
- 3.4 Selection Structure
- 3.5 Iteration structures
 - 3.5.1 While statement
 - 3.5.2 for statement
 - 3.5.3 Video 3: Control Structures in Python
 - 3.5.4 break, continue and pass Statements
- 3.6 Data Structures
 - 3.6.1 Video 4: Working with Data Types and Data Structures
 - 3.6.2 List
 - 3.6.3 Video 5: Operators and Control Structures in Python
 - 3.6.4 Range Type
 - 3.6.5 Tuples
 - 3.6.6 Dictionaries
- 3.7 Summary
- 3.8 References and Further Reading
- 3.9 Answer to Check your progress

3.1 INTRODUCTION

In a program, a control structure is a block of code that determines the order in which the statements are executed. It also directs the flow of the program based on the given logic. There are two key control structures, namely, selection and iteration to structure the segments of code. Selection statements allow a programmer to write statements that will be executed based on the satisfaction of some condition while iteration statements allow the repetition of the same set of statements multiple times.

A data structure is a particular way of organising data in a computer that enables efficient use. In this unit we will also discuss common data structures in Python.

3.2 OBJECTIVES

Upon completion of this unit you will be able to:

- *explain* the use of control structures and data structures in a program.
- *identify* appropriate control structures and data structures for a given scenario.
- *apply* suitable data structures to model a solution for a simple problem.

3.3 TERMINOLOGIES

iterative: repeating

slice: part of a data structure

string: a special data type

3.4 SELECTION STRUCTURE

As in many other languages, Python provides an *ifelse* statement. the general syntax of the *if ... else* statement is;

if <condition>:

<Statements to be run if the condition evaluates to true>

else:

<Statements to be run if the condition evaluates to false>

For example, the following program prints a warning message if the weight of a customer's luggage exceeds the allowed limit of 25 kgs.

Example 3.1:

```
allowedlimit = 25
extraweight=0
weight = float(input("How many kilograms does your" "baggage
weigh? "))
if weight > 25:
    extraweight = weight - allowedlimit
    print('Your luggage has ',extraweight,'kgs extra')
    print('There will be an extra chrage of',extraweight*10)
else:
    print("Your luggage weight is within the limit")
print("Thank you for your business.")
```

The above program checks if the weight is above the allowed limit. If it above the limit, it informs the customer the extra payment involved. Otherwise, it will inform that the weight is within the limit. Else clause is used to execute involved in the 'otherwise' part of the condition.

If statements can be embedded within one another and are called nested if condition. The level of nesting depends on the logic of the program and you can have any number of nested statements. For example, the following

modifies the example 3.1 to allow customers with a small extra weightage to go through without a payment.

Example 3.2

```
allowedlimit = 25
extraweight=0
weight = float(input("How many kilograms does your baggage weigh?
"))
if weight > allowedlimit:
    extraweight = weight - allowedlimit
    if extraweight < 5:
        print("Your luggage has ", extraweight, "kgs extra")
        print("Since it is a small value, we will not charge you this
time")
    else:
        print("Your luggage has ", extraweight, "kgs extra")
        print("There will be an extra charge of ",extraweight*10,"\n")
else:
    print("Your luggage weight is within the limit")
    print("Thank you for your business.")
```

In some cases, we need to do things based on multiple tests. For example, let us say a teacher decided to give grades to her students as follows:

Mark greater than or equal to 80 Excellent

Mark greater than or equal to 65 but less than 80 Good

Mark greater than or equal to 50 but less than 65 Pass

Mark less than 50 Fail

We can write a program to print a grade according to a student's mark with multiple if statements as given in example 3.3a.

Example 3.3 a

```
#Program: Printing grades    mark =
float(input("Enter the
students mark"))
if mark >= 80:
    grade = "Excellent" else:
# check grades for the condition Good
    if mark >= 65:
        grade = "Good"
    else:
# check grades for the condition Pass
        if mark >= 50:
            grade = "Pass"
        else:
# check grades for the condition Fail
            grade = "Fail"
print("Your grade for this"
"subject is:", grade)
```

Example 3.3 b

```
#Program: Printing grades
mark = float(input("Enter the
students mark"))
if mark >= 80:
    grade = "Excellent"
elif mark >= 65:
    grade = "Good"
elif mark >= 50:
    grade = "Pass"
else:
# grade has to be fail
    grade = "Fail"
print("Your grade for this"
"subject is:", grade)
```

Python provides another statement called elif, which allows us to write more compact code with less indentation as given in example 3.3b. elif is a short hand for else if. In all of the above else part is optional.

3.5 ITERATION STRUCTURES

Python provides multiple statements to implement iteration, i.e. repeating the same set of instructions with different value sets. **while** statement and **for** statement are two such key statements.

3.5.1 While statement

In a while statement executes a set of instructions if a given condition is true. It takes the form of

```
while <condition>
    <set of statements>
```

Only if the condition is true, then the set of statements is executed. This is how the while loop operates:

Step 1: Evaluate the condition

Step 2: If the *condition is false* continuing the program with the next statement that comes after the set of statements within while condition

Step 3: If the *condition is true* execute the set of statements within

while condition

Step 4: Go back to step 1 and evaluate the condition again

The following is an example of the use of the while loop: It prints numbers from 1 to 9 (last number not less than 10). The condition $n < 10$ will be true for all numbers from 0 to 9. the condition will become false when n becomes 10, because 10 is not less than 10).

```
#while loop n = 0
while n < 10:
    n = n + 1
    print(n)
print('All numbers printed')
```

As you can see, it is important that when the condition is true, it changes sometime during the execution of the set of statements. Otherwise, it will run for ever (known as an infinite loop). In the above example, if we forget to increase n by 1 during each iteration, it will become an infinite loop.

3.5.2 for statement

Unlike other languages, Python for statement is not based on a counter variable. Instead, for loop iterates through a set of values given as a list or a string. The syntax of the for loop is

```
for <item> in <list>:
    <do something with the item>
```

The loop will iterate through the list or the string assigning one item or character to the control variable, then executing the block of statements using that value.

```
import math
numberlist = [1,2,3,4,5,6]
for targetvalue in numberlist:
    print(targetvalue, "squared is",int(math.pow(targetvalue,2)))
```

In the example, control variable is named *targetvalue*. Starting with the first value in the list, it will take values in the list in the given sequence. In this example, it will take values 1,2,3,4,5 and 6 in respectively. During each iteration it will print the square value of the control variable.

3.5.3 VIDEO 3: CONTROL STRUCTURES IN PYTHON

You may watch this video on Control Structures in Python

Before attempting the Activity 3.1.

URL: <https://youtu.be/5ogPFf8wGw>



Activity 3.1

Write a program using a while loop to execute even numbers starting from 20 to 60.

3.5.4 break, continue and pass Statements

The `break` statement ends execution of the nearest enclosing loop or conditional statement in which it appears. It can be used to terminate or change the ongoing iteration. `break` statement is commonly used with the looping statements. In while and for loops, it terminates the nearest enclosing loop, skipping the optional `else` clause if the loop has one. If a `for` loop is terminated by `break`, the loop control target keeps its current value. When `break` passes control out of a `try` statement with a `finally` clause, that `finally` clause is executed before really leaving the loop.

Another useful statement is the `continue` statement. When the `continue` statement is found, It continues with the next cycle of the nearest enclosing loop. When `continue` passes control out of a `try` statement with a `finally` clause, that `finally` clause is executed before really starting the next loop cycle.

Some examples of the use of `break` and `continue` statements in while and for loops are shown below.

<pre>n = 0 while n < 10: n = n + 1 print(n) if n == 5: break print('All numbers printed')</pre>	<pre>#while loop n = 0 while n < 10: n = n + 1 if n % 2 == 0: continue else: print(n)</pre>
Print all numbers from 1 to 4. Though the condition to check is n < 10, when n is 5 the break statement will go to the next statement after the while statement.	This program will only print odd numbers. When an even number is found, it will go to the next iteration of the loop, starting at the while statement.

This program will print only numbers from 1 to 4. When n becomes 5 it will finish the current while statement and run the next statement which is `print('All numbers printed')`.

This program will print numbers from 1 to 10 only if the number is odd. When the number is even (i.e. the remainder when the number is divided by 2 is 0), it goes to the next iteration. When n becomes 5 it will finish the current while statement and run the next statement which is `print('All numbers printed')`.

There is one other statement known as the pass statement which is used when the body segment in a program is empty. That is, when a statement is required to be written so that the program syntax is correct, but no action is

```
if x < 0:
    pass # need to handle negative values!
```

required. In such cases a pass statement is used as follows.

or

```
>>> while True:
...     pass # Busy-wait for keyboard interrupt (Ctrl+C)
... 
```

3.6 DATA STRUCTURES

Python has a rich repertoire of data structures that allow the manipulation of sets of data easily and efficiently. Here we discuss some of the basic data structures such as List, Tuple, Dict.

At this point, it is useful to understand two types of data: **immutable** and **mutable**. Immutable data are values that cannot be changed once created. - For example, int, float, long, str, tuple etc are immutable data types whereas list, set and dict are mutable data types. It is important to understand that only mutable objects support methods that change the object in place, such as reassignment of a sequence slice, which will work for lists, but raise an error for tuples and strings.

VIDEO 4: WORKING WITH DATA TYPES AND DATA STRUCTURES

You may watch this video with a screen cast to see how to work with Python Data Types.

URL: <https://youtu.be/1lwY4eaI-0o>



A slice is a part of a data item such as a list and the range of the slice is marked using the notation [start index: stop index]. Slice operator (:) also has an extended version that includes a stride parameter as follows: [start index: stop index: stride]. The stride value is similar to the step value in range statement.

Example 3.4

```
# Slicing lists
characters =
['a','b','c','d','e','f','g','h','i','j','k','l','m','n']
simpleslice = characters[3:8]
print('Some characters are: ', simpleslice)
slicewithstride = characters[3:8:2]
print('Slicing with stride parameter:',slicewithstride)

output
Simple slice:  ['d', 'e', 'f', 'g', 'h']
Slicing with stride paramter:  ['d', 'f', 'h']
```

In the above, the Simple slicing extracts the characters from elements at index 3 to element at 7 (upper index - 1). Remember that indexing starts with 0.

Slicing with strid parameters extracts every 2nd character from elements at index 3 to element at 7 (upper index - 1).

Since these parameters are optional, omitting some characters will work as follows:

`characters[start:end] # items start through end-1`

Table 3.1 slicing operator examples with a list

<code>characters[start:]</code>	items start through the rest of the array
<code>characters[:end]</code>	items from the beginning through end-1
<code>characters[:]</code>	a copy of the whole array
<code>characters[-1]</code>	last item in the array. - sign indicates extracting from the end of the list.
<code>characters[-2:]</code>	last two items in the array
<code>characters[:-2]</code>	everything except the last two items

The following list of operations with mutable objects is given in official Python documentation.

Table 3.2 list of operations with mutable objects

<code>s[i] = x</code>	item <code>i</code> of <code>s</code> is replaced by <code>x</code>
<code>s[i:j] = t</code>	slice of <code>s</code> from <code>i</code> to <code>j</code> is replaced by the contents of the iterable <code>t</code> of the same size
<code>del s[i:j]</code>	same as <code>s[i:j] = []</code>
<code>s[i:j:k] = t</code>	the elements of <code>s[i:j:k]</code> are replaced by those of <code>t</code>
<code>del s[i:j:k]</code>	removes the elements of <code>s[i:j:k]</code> from the list
<code>s.append(x)</code>	appends <code>x</code> to the end of the sequence (same as <code>s[len(s):len(s)] = [x]</code>)
<code>s.clear()</code>	removes all items from <code>s</code> (same as <code>del s[:]</code>)
<code>s.copy()</code>	creates a shallow copy of <code>s</code> (same as <code>s[:]</code>)
<code>s.extend(t)</code> or <code>s += t</code>	extends <code>s</code> with the contents of <code>t</code> (for the most part the same as <code>s[len(s):len(s)] = t</code>)
<code>s *= n</code>	updates <code>s</code> with its contents repeated <code>n</code> times
<code>s.insert(i, x)</code>	inserts <code>x</code> into <code>s</code> at the index given by <code>i</code> (same as <code>s[i:i] = [x]</code>)
<code>s.pop([i])</code>	retrieves the item at <code>i</code> and also removes it from <code>s</code>
<code>s.remove(x)</code>	remove the first item from <code>s</code> where <code>s[i] == x</code> . Raises an error when the value is not found.
<code>s.reverse()</code>	reverse the items of <code>s</code> in place. Note that it does not return the reversed list.

These operations are applicable to mutable data types.

3.6.1 List

A list is a data structure that can represent an ordered set of items. Lists are said to mutable in that the size of the list and the items within it can be changed after creating a list.

The format for a list is a set of items enclosed by square brackets `[ ]`.

Example 3.5

```
List of strings:['Sri Lanka', ' India', 'Pakistan',
'bangaladesh', 'Nepal']

List of numbers:[1,3,5,7,11,13]

A list of mixed items:['Saman', 45, '-34.67', 'kumari']

The empty list:[]
```


We have already seen how we can iterate through values in a list using a `for` statement. Other operations that can be done on lists were described in the previous section under Data Structures.

VIDEO 5: OPERATORS AND CONTROL STRUCTURES IN PYTHON

You may watch this video on Operators and Control Structures in Python before attempting the Activity 3.2.

URL: <https://youtu.be/qxw690359YY>



Activity 3.2

Write a program using a `for` loop to print the cube value (power of 3) of five numbers given in a list.

3.6.2 Range Type

Range is a built-in type that returns an arithmetic sequence. In versions prior to version 3, `range` was a function that returned a list. `Range` generates a list of numbers up to, but not including, the stop value in the range statement. The general formats of `range` are;

```
range(stop).  
range([start], stop [,step]) : where values for  
[start] and [,step] are optional.
```

All values for `start`, `stop` and `step` must be positive or negative integers. If value for `start` is not specified, the default value of 0 is assigned. For `step` the default value is 1.

For example:

```
range(5) will generate 0,1,2,3,4  
range(5,10) will generate 5,6,7,8,9  
range(0,10,3) will generate 0,3,6,9
```

`range` type is commonly used in `for` loops to iterate through a known sequence of values.

Example 3.6: Repeat a sequence 10 times, the following uses the `range` type to generate numbers from 0 to 9 and then print each one.

```
for item in range(10):  
    # range will generate values from 0 to 9 print(item)
```

Example 3.7 : To iterate over a list using the index value of items in the list

```
cities = ['Colombo', 'Kandy', 'Jaffna', 'Matara',  
          'Anuradhapura']  
for item in range(len(cities)):  
    # range will use the length of the list  
  
    print('City in position ', item, ' of the list is ',  
          cities[item])
```

If you want to convert the arithmetic progression generated by range to a list, a conversion can be used.

Example 3.8 : Convert values generated by range into a list of value

```
valuelist = list(range(5))  
Now, valuelist = [0,1,2,3,4]
```

3.6.3 Tuples

Tuples are similar to lists in that they collect related data together. A tuple is of fixed size and is immutable, i.e. cannot be changed once created. Tuples are useful when you want to create a record structure of related data items. Tuples are enclosed within brackets. For example,

```
aruna = ('Aruna', 'Silva', 1978, '747872567').
```

You can use indexes to select items from a tuple as in a list or a string. for example aruna[2] will return 1978.

Tuples are immutable objects, so you cannot change the values of a tuple, for example, the following statement is illegal.

```
aruna[2] = 1979
```

However, we can create a new tuple using the values from the existing tuple, and adding some extra values given as a tuple

for example,

```
aruna = aruna[0:1] + (1979,) + aruna[3:]
```

will assign the tuple ('Aruna', 'Silva', 1979, '747872567') to the variable aruna. Note that a tuple with a single value has to be ended with a comma after the value.

One of the very powerful operation with tuples is the tuple assignment. It allows values in a tuple given in the right side of the assignment operator to be assigned to a set of variables given in the left.

for example

```
(firstname, lastname, birthyear, phone) = aruna
<will do the following assignments>
firstname = 'Aruna', lastname = 'Silva', birthyear =
1979, phone = '747872567'
```

Remember that the number of variables on the left side should be the same as the number of values in the tuple given in right.

Activity 3.3

Write a program to create a list with a car details and display them.

3.6.4 Dictionaries

Dictionaries are another useful data structure. It consists of a set of

key: value pairs enclosed within curly brackets {}. For example, the following is a list of telephone numbers:

```
phonebook = {'Saman':112847365,
'Iresha':772726286, 'Aziz':1124273254,
'Bhavani':717976475}
```

We use an index to access value from a list, string or a tuple, values of

Access a value from a dict	phonebook['Aziz']
New values can be assigned to a dict	phonebook['Harry'] = 779274946
Values from a dict can be deleted with del	del phonebook['Saman']
Keys and values can be assigned to lists	names = list(phonebook.keys()) phonenos = list(phonebook.values())
Check if a key is in the dict	'Iresha' in phonebook:
Use an expression to create a dict	(number:number** for number in (1,2,3,4,5))
Both key and value from a dict can be retrieved with a loop using the items() method of dict	for key,value in phonebook.items(): print(key,value)

Table 3.3 Way to access values from a list, string or tuple

Check Your Progress

- Q-1 Write a program in Python to place a sports person in the right category Viz. Beginner Level, Middle Level, Expert Level; as per the performance score of the person entered by the Coach.

- Q-2 Compare the break , continue and pass statements, give suitable example of each
- Q-3 What are mutable data structures in Python? How do they differ from the immutable data structures? List all mutable and immutable data structures in Python.

3.7 SUMMARY

In this unit you learned the control structures that determine the execution of a program. We discussed the two main control structures, selection and iteration providing appropriate examples on how to program in Python. We also discussed the manipulation of simple data structures that is available in Python

3.8 FURTHER READING

- 1) 1. Allen B. Downey (2012). Think Python.
<http://greenteapress.com/wp/think-python/>
- 2) <https://www.ibiblio.org/g2swap/byteofpython/read/index.html>

3.9 ANSWER TO CHECK YOUR PROGRESS

- Ans-1 Refer section 3.4
- Ans-2 Refer section 3.5.4
- Ans-3 Refer section 3.6